

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 5 de Rust

Web, Concurrencia y Juegos

Recomendado a partir de los 19-20 años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. Introducción	1
2. El Universo Web	2
2.1. ¿Cómo funciona la Web? El protocolo HTTP	2
2.2. El Protocolo HTTP y Tu Primer Servidor	2
2.3. ¿Cómo viaja la información por la web?	2
2.4. Peticiones (Requests) y Respuestas (Responses)	3
2.5. Proyecto Práctico: Creando un Mini-Servidor Web desde cero en Rust	4
3. Concurrencia Segura y Asincronía en Rust	8
3.1. La relación entre Concurrencia y Asincronía en Rust	8
3.2. ¿Cómo funciona async / await en Rust?	10
3.2.1. ¿Qué es un Trait?	10
3.2.2. A. El Trait Future (El modelo de «Tirar» o Poll)	11
3.2.3. B. Qué hace async por dentro: La Máquina de Estados	12
3.2.4. C. El Rol del Runtime (Ej. Tokio o el de Macroquad)	12
3.3. Ejemplo práctico: Integración en nuestro videojuego	13
3.4. Concurrencia Segura - El Superpoder de Rust	13
3.5. ¿Qué es la concurrencia y por qué es difícil? El problema de los hilos de ejecución.	13
3.6. 2.3. Creando hilos (std::thread)	14
3.7. El enfoque de Rust: Fearless Concurrency (Concurrencia sin miedo).	14
3.8. Creando y gestionando hilos (std::thread)	15
3.9. Compartiendo datos de forma segura: Canales de comunicación (mpsc)	16
3.10. 2.4. Canales de comunicación (mpsc)	17
4. Proyecto Final de Ciclo - Tu Primer Videojuego Gráfico	18
4.1. Introducción al desarrollo de videojuegos con Macroquad.	18
4.2. El bucle principal de un juego (Game Loop)	18
4.3. Desarrollo Guiado: Clon de Pong / Rust-Bird	19
4.4. 3.3. Desarrollo Guiado: Un clon de Pong simplificado	21
5. Ejercicios propuestos	22
6. Desafíos propuestos para el alumno (Personalización e integración)	22

1. Introducción

¡Bienvenido al último paso de tu viaje con Rust! En los cuadernos anteriores has aprendido a controlar la memoria, a estructurar tus datos y a gestionar errores como un profesional. En este cuaderno vas a aplicar todo eso para construir aplicaciones que interactúan con el mundo exterior: páginas web que tú mismo servirás, un programa que gestiona la concurrencia y un videojuego interactivo con gráficos en tiempo real.

Hablar de:

- Videojuego
- Servidor web
- ¿Concurrencia?

2. El Universo Web

El desarrollo moderno de software no se puede entender sin su conexión con la red, y es aquí donde el protocolo HTTP se consolida como el idioma universal para la comunicación entre clientes y servidores. A lo largo de este capítulo, exploraremos los fundamentos de cómo funciona la Web y cómo viaja la información a través de la infraestructura global de internet. Analizaremos en detalle la anatomía de las peticiones (Requests) y respuestas (Responses) que componen cada interacción digital, para finalmente consolidar todo este conocimiento teórico en un proyecto práctico de ingeniería: la construcción y programación de un mini-servidor web desde cero utilizando Rust, aprovechando su velocidad y seguridad de memoria nativas.

2.1. ¿Cómo funciona la Web? El protocolo HTTP

Estamos acostumbrados a viajar por la web de forma intuitiva colocando una dirección en el campo de URL del navegador o haciendo clic en los enlaces de las páginas web. Sin embargo, cada vez que entras en una página web, tu navegador (cliente) envía una Petición (Request) a un ordenador remoto (servidor). El servidor lee esa petición y devuelve una Respuesta (Response) que suele contener texto en formato HTML, imágenes o datos.

El protocolo HTTP es el idioma estándar en el que se comunican cliente y servidor. Una petición básica le dice al servidor: «Quiero obtener (GET) el archivo principal (<https://www.google.com/>)». El servidor responde con un código de estado (como 200 OK si todo fue bien, o el famoso 404 Not Found si no existe el archivo) seguido del contenido de la página.

2.2. El Protocolo HTTP y Tu Primer Servidor

El protocolo HTTP (Hypertext Transfer Protocol) es la base de la comunicación en la World Wide Web, permitiendo que un cliente (como un navegador) y un servidor intercambien datos de forma estructurada. En el ecosistema de Rust, crear un primer servidor web implica abrir **un puerto TCP** para escuchar conexiones entrantes, **procesar los bytes recibidos** según las reglas de HTTP y **devolver una respuesta** adecuada. Para proyectos iniciales, esto se puede lograr utilizando la biblioteca estándar (`std::net::TcpListener`), lo que permite entender los fundamentos de la red antes de dar el salto a frameworks (marcos de trabajo) más avanzados del ecosistema como **Axum** o **Actix-web**.

2.3. ¿Cómo viaja la información por la web?

La información en la web viaja empaquetada en mensajes de texto plano a través de la pila de protocolos TCP/IP, donde HTTP opera en la capa de aplicación para definir el formato de dichos mensajes. Cuando un usuario introduce una URL, el navegador realiza una resolución DNS para encontrar la dirección IP del servidor, establece una conexión segura mediante TLS/SSL (HTTPS) y transmite la información fragmentada en paquetes de red. Al ser HTTP un protocolo sin estado (stateless), cada interacción se trata de forma independiente, lo que significa que el servidor no recuerda las peticiones anteriores a menos que se utilicen mecanismos adicionales como cookies o tokens de sesión.

2.4. Peticiones (Requests) y Respuestas (Responses)

Una petición (Request) HTTP es el mensaje que envía el cliente e incluye obligatoriamente un método o verbo de los siguientes (GET, POST, PUT, DELETE), una URL o ruta, la versión del protocolo, cabeceras (headers) con metadatos y, opcionalmente, un cuerpo (body) con datos.

Por tanto, cuando escribes una dirección web en el navegador o pinchas sobre un enlace, estás mandando al servidor una petición (Request) similar a la siguiente:

```
GET /index.html HTTP/1.1
Host: //mi-servidor-rust.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html
Connection: close
```

Hemos supuesto que mandas la petición al servidor: `mi-servidor-rust.com`

- Línea 1: Indica la acción (GET, para obtener datos), el archivo solicitado (/index.html) y la versión del protocolo (HTTP/1.1).
- Líneas 2-5: Son las cabeceras (metadatos). Le dicen al servidor a qué dominio se conecta (Host), qué navegador se usa (User-Agent) y qué tipo de archivo espera recibir (Accept).

Por otro lado, la respuesta (Response) es la contestación del servidor, la cual contiene un código de estado numérico (como 200 OK, 404 Not Found o 500 Internal Server Error), cabeceras que describen el contenido (como Content-Type) y el cuerpo que alberga el recurso solicitado (HTML, JSON, imágenes). En Rust, estos componentes se modelan mediante estructuras de datos y enums que garantizan un tipado seguro, evitando errores comunes al parsear o construir mensajes web.

En el caso de la petición anterior, la respuesta del servidor podría ser la siguiente:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 114
Connection: close

<html>
  <body>
    <h1>¡Hola Mundo desde Rust!</h1>
    <p>Tu primer servidor funciona.</p>
  </body>
</html>
```

- Línea 1: La línea de estado. Confirma que todo ha ido bien devolviendo el código numérico 200 OK.
- Líneas 2-4: Las cabeceras de respuesta. Informan al navegador que lo que va a leer es un archivo web (text/html) y cuánto mide exactamente en bytes (Content-Length).
- Línea 5: Una línea en blanco obligatoria. Es el separador crítico que le dice al navegador: «Fin de los metadatos, empieza la página web».
- Bloque final: El cuerpo (Body). El código HTML puro que el navegador interpretará para pintarlo de forma visual en la pantalla del usuario.

2.5. Proyecto Práctico: Creando un Mini-Servidor Web desde cero en Rust

Para entenderlo de forma nativa, utilizaremos la biblioteca estándar de Rust (`std::net`) para escuchar conexiones en nuestra máquina y responder con una página web real. Código de ejemplo (`src/main.rs`):

Para construir un mini-servidor web nativo en Rust sin dependencias externas (crates), lo que llamamos de forma nativa, se utiliza el módulo `std::net::TcpListener` para escuchar conexiones en nuestra máquina y responder con una página web real. También nos permitirá enlazar el programa a una dirección local y un puerto, como por ejemplo **127.0.0.1:8080**.

El servidor se diseña mediante un bucle continuo que acepta **flujos de datos entrantes** (`TcpStream`), lee los bytes de la petición en un buffer de memoria y los procesa para identificar la ruta solicitada.

❗ Nota: Un buffer de memoria es un espacio de almacenamiento temporal donde se guardan datos de forma provisional mientras se transfieren de un lugar a otro. Funciona como una sala de espera o `una caja de cartón` donde el programa va acumulando los bytes que llegan desde el exterior (como una petición web) hasta que tiene la cantidad suficiente para procesarlos todos juntos de forma eficiente.

Finalmente, el código evalúa la solicitud y utiliza el método `stream.write_all` para enviar de vuelta una cadena de texto con formato analizable por el navegador, completando así el ciclo de vida de una transacción web con la máxima eficiencia y seguridad de memoria que caracteriza a Rust.

Código de ejemplo (`src/main.rs`):

```
use std::io::{prelude::*, BufReader};
use std::net::{TcpListener, TcpStream};

fn main() {
    // 1. Escuchamos en la dirección local (localhost) en el puerto 8080
    let listener = TcpListener::bind("127.0.0.1:8080").unwrap();
    println!("Servidor web iniciado en http://127.0.0.1:8080");

    // 2. Esperamos y procesamos las conexiones de los clientes
    for stream in listener.incoming() {
        let stream = stream.unwrap();
        gestionar_conexion(stream);
    }
}

fn gestionar_conexion(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&mut stream);

    // Leemos la primera línea de la petición (ej. "GET / HTTP/1.1")
    let linea_peticion = buf_reader.lines().next().unwrap().unwrap();
    println!("Petición recibida: {}", linea_peticion);

    // Definimos la respuesta HTTP con código 200 OK y contenido HTML
```

```

let contenido_html = r#"
    <!DOCTYPE html>
    <html lang="es">
    <head>
        <meta charset="UTF-8">
        <title>Mi Servidor en Rust</title>
        <style>
            body { font-family: sans-serif; background-color: #f4f4f9; text-align:
center; padding-top: 50px; }
            h1 { color: #df4b32; }
        </style>
    </head>
    <body>
        <h1>¡Hola desde mi servidor Rust!</h1>
        <p>Este servidor ha sido programado desde cero en 2º de Bachillerato.</p>
    </body>
    </html>
"#;

let longitud = contenido_html.len();
let respuesta = format!(
    "HTTP/1.1 200 OK\r\nContent-Length: {}\r\n\r\n{}",
    longitud, contenido_html
);

// Enviamos la respuesta de vuelta al navegador del usuario
stream.write_all(respuesta.as_bytes()).unwrap();
}

```

Explicación

1) En el main, la línea:

```

// 1. Escuchamos en la dirección local (localhost) en el puerto 8080
let listener = TcpListener::bind("127.0.0.1:8080").unwrap();

```

Crea el servidor (listener) y queda escuchando en la dirección local del ordenador (127.0.0.1) y el puerto (8000). Habría sido equivalente escribir (localhost:8000). Solo podrás lanzar peticiones al servidor desde tu propio ordenador utilizando la dirección y puerto indicados anteriormente en cualquiera de las dos formas.

2) El bucle for:

```

// 2. Esperamos y procesamos las conexiones de los clientes
for stream in listener.incoming() {
    let stream = stream.unwrap();
    gestionar_conexion(stream);
}

```

¿Que es un stream?

Un stream (o flujo de datos) es como una tubería digital continua que conecta dos puntos (por ejemplo, tu navegador tu servidor en Rust) para transportar información de manera constante, bit a bit y en un orden estricto.

La analogía de **YouTube**: Cuando ves un vídeo en **streaming**, no esperas a que se descargue la película entera de 2 GB para empezar a verla; el vídeo te va llegando como un

flujo constante de datos que reproduces en tiempo real mientras el resto sigue viajando por la red.

¿Qué resulta ser **listener.incoming()**?

`listener.incoming()` es un iterador (una secuencia infinita) que se queda «escuchando» el puerto de red. Cada vez que un cliente (como un navegador) intenta conectarse a nuestro servidor, este iterador atrapa esa conexión y nos da acceso a ella.

- Cómo funciona en la práctica: Imagínalo como una cinta transportadora en una fábrica. El bucle `for` se detiene y se queda esperando. En el momento en que alguien entra en la web, la cinta se mueve y nos entrega un nuevo paquete con la conexión.
- El truco de Rust: Este iterador nunca termina por sí solo. Mantiene el servidor vivo y funcionando en un bucle continuo para recibir visitas una detrás de otra.

¿En qué se traduce **`let stream = stream.unwrap()`**?

Esta línea realiza dos acciones fundamentales de Rust: desempaquetar el resultado seguro y redefinir la variable.

A. El desempaqueado con `.unwrap()`

En Rust, una conexión de red puede fallar en el último milisegundo (por ejemplo, si el cliente pierde la conexión justo cuando el servidor la estaba aceptando). Por eso, el iterador no te da la conexión directamente, sino que te da un envoltorio de seguridad llamado `Result<TcpStream, Error>`.

`.unwrap()` significa: «Confío en que la conexión está bien. Abre el envoltorio, dame el flujo de datos real (`TcpStream`) para que pueda trabajar con él y, si hubo un error crítico, detén el programa».

Es decir, si realmente ha habido un error en la conexión, tal como lo tenemos programado nosotros el programa fallaría y daría un (`panic!`), el programa se colgaría.

B. La redefinición (`let stream = ...`)

Esto se conoce en Rust como Variable Shadowing (Sombreado de variables). Permite reutilizar el mismo nombre (`stream`) de una variable para crear otra nueva variable.

- El primer `stream` (el del `for`) es el envoltorio con el posible error.
- El segundo `stream` (el del `let`) es la conexión limpia, extraída y lista para usar en la función `gestionar_conexion`.

La función `gestionar_conexion`

La función `gestionar_conexion` es el cerebro operativo de nuestro servidor. Es la encargada de recibir la «tubería» (`stream`) que abrió el cliente, leer lo que nos están pidiendo, y escribir la respuesta HTTP de vuelta para que el navegador pueda pintar la página web.

Aquí tienes el código típico de esta función en Rust explicado paso a paso:

```
use std::io::prelude::*; // Importa los traits Read y Write para poder usar leer y
                          escribir en el stream
use std::net::TcpStream;
```

```
fn gestionar_conexion(mut stream: TcpStream) {
    // 1. Creamos el buffer de memoria (la sala de espera temporal)
    // Una matriz de 1024 bytes llenos de ceros
    let mut buffer = [0; 1024];

    // 2. Leemos los bytes que viajan por el stream y los metemos en el buffer
    stream.read(&mut buffer).unwrap();

    // 3. Preparamos la respuesta HTTP correcta (Línea de estado + Cabecera + Cuerpo)
    let contenido = "<html><body><h1>¡Servidor Rust Funciona!</h1></body></html>";
    let respuesta = format!(
        "HTTP/1.1 200 OK\r\nContent-Length: {}\r\n\r\n{}",
        contenido.len(),
        contenido
    );

    // 4. Enviamos la respuesta de vuelta por la tubería convirtiendo el texto en bytes
    stream.write_all(respuesta.as_bytes()).unwrap();

    // 5. Aseguramos que todos los bytes salgan de la memoria hacia la red
    stream.flush().unwrap();
} // <-- Aquí la variable 'stream' se destruye automáticamente y la conexión se cierra de
forma segura
```

Explicación del paso a paso para la clase:

1. `mut stream: TcpStream`: La variable debe ser mutable (`mut`) porque leer y escribir datos altera el estado interno de la tubería (los bytes entran y salen, modificando su posición).
2. El Buffer (`[0; 1024]`): Creamos una «caja» vacía con capacidad exacta para 1024 bytes. Es el espacio temporal donde volcaremos el texto de la petición del cliente.
3. `stream.read(...)`: El servidor mete la mano en la tubería, saca los bytes que envió el navegador y los deposita en nuestro buffer.
4. La construcción de respuesta: Creamos un String con la estructura exacta que exige el protocolo HTTP (que vimos en los apartados anteriores). Usamos `\r\n` (las dos líneas en blanco obligatorias) para separar los metadatos del código HTML real.
5. `stream.write_all(...)`: Transforma nuestro texto a bytes binarios (`.as_bytes()`) y los empuja a través de la tubería de regreso al navegador del usuario.
6. `stream.flush(...)`: Fuerza al sistema operativo a enviar inmediatamente cualquier byte rezagado que haya quedado esperando en la memoria intermedia de la tarjeta de red.

Al llegar al cierre de la llave `}`, se aplica la magia del Ownership (Propiedad) de Rust: la variable `stream` sale de su ámbito (scope), el compilador libera el recurso y la conexión de red se cierra automáticamente sin riesgo de fugas de memoria.

Al ejecutar este programa y abrir <http://127.0.0.1:8080> en cualquier navegador, verás tu página web. El servidor lee la petición, monta el mensaje HTTP correcto y lo envía de vuelta.

3. Concurrencia Segura y Asincronía en Rust

Todos los programas que hemos diseñado en los cuadernos anteriores comparten una característica fundamental: son estrictamente síncronos y no aplican concurrencia. Esto se traduce en que, al ejecutarse, consumen un único hilo del microprocesador. Sin embargo, la informática actual ha cambiado. La gran mayoría de los procesadores modernos disponen de varios núcleos de hardware, y los sistemas operativos son capaces de gestionar miles de hilos de software de forma simultánea. Para aprovechar este potencial, los lenguajes modernos incorporan recursos que permiten a los programas exprimir cada núcleo de la CPU.

Algunas definiciones

1. **Sobre el concepto de monohilo (Síncrono):** Un programa síncrono tradicional se ejecuta en un único hilo de software procesando las instrucciones en orden secuencial. El sistema operativo asigna ese hilo a un solo núcleo de la CPU a la vez. Si ese núcleo está al 100% de su capacidad o se queda esperando a que el disco duro lea un archivo, todo tu programa se congela por completo en ese instante.
2. **Hilos de Software vs. Núcleos de Hardware:** Los sistemas operativos son tan avanzados que pueden gestionar miles de hilos de software al mismo tiempo, incluso si el microprocesador solo tiene, por ejemplo, 8 núcleos físicos de hardware. El sistema operativo va rotando esos hilos en los núcleos tan rápido (en microsegundos) que nos da la ilusión de que todo se ejecuta a la vez. Rust te da el control total para crear estos hilos de software de forma 100% segura.
3. **Concurrencia vs. Asincronía:** La concurrencia (hilos) divide el trabajo pesado entre los núcleos de la CPU. La asincronía (tareas/Futures) optimiza los tiempos muertos de espera (red, discos) para que un solo hilo pueda hacer otras cosas mientras espera, logrando hacer «más cosas al mismo tiempo» pero con una filosofía y un coste radicalmente distintos.

En el ecosistema de Rust, la concurrencia y la asincronía son las dos herramientas clave para hacer más de una cosa al mismo tiempo; aunque, como descubriremos a continuación, operan bajo filosofías, costes y mecanismos de ejecución completamente diferentes.

3.1. La relación entre Concurrencia y Asincronía en Rust

Para entender cómo se relacionan, primero debemos diferenciar sus conceptos:

- **Concurrencia (Basada en Hilos / Threads):** Está orientada a la CPU. El sistema operativo levanta hilos nativos (`std::thread`). Cada hilo tiene su propia pila de memoria (stack) asignada por el sistema (normalmente unos 2 MB). Si tienes 4 tareas pesadas de cálculo matemático, creas 4 hilos distribuidos en los núcleos de tu procesador. Su coste de conmutación (cambiar de un hilo a otro) es gestionado por el hardware/S.O. y es relativamente alto.
- **Asincronía (Basada en Tareas / Futures):**

Está orientada a operaciones de Entrada/Salida (I/O), como esperar una respuesta de red (HTTP), una consulta a la base de datos o la lectura de un archivo. En lugar de bloquear un hilo entero del sistema operativo mientras esperas los bytes, el hilo «suelta» la tarea y se dedica a procesar otra y por tanto permanece activo.

La conexión entre Concurrencia y Asincronía: Múltiples tareas sobre pocos hilos

La asincronía en Rust es un modelo de concurrencia cooperativa. Te permite ejecutar miles o millones de tareas simultáneas (comportamiento concurrente) utilizando un número mínimo de hilos del sistema operativo (a menudo, tantos hilos como núcleos tenga tu CPU).

Característica	Concurrencia Clásica (<code>std::thread</code>)	Asincronía (<code>async/await</code>)
Enfoque principal	Tareas intensivas de CPU (Cálculo).	Tareas intensivas de I/O (Red, Discos).
Coste de memoria	Alto (Megabytes por hilo del S.O.).	Prácticamente cero (Bytes por tarea).
Gestor	El Sistema Operativo (Planificador).	Un <i>Runtime</i> en espacio de usuario (ej. Tokio).
Bloqueo	Bloquea el hilo por completo.	Pausa la tarea, el hilo sigue libre.

La tabla anterior merece algunos comentarios avanzados que posiblemente no necesitarás para entender el código de los programas que vienen a continuación pero reafirmarás las bases que estas aprendiendo.

1. Coste de memoria: ¿Por qué vemos en la tabla MB por hilo vs. Bytes por tarea? ¿No depende de la tarea a realizar?

Sí depende de la tarea, pero la diferencia radical está en el peaje de entrada que te cobra el sistema operativo por el simple hecho de existir.

- **Concurrencia Clásica (`std::thread`):** Cuando le pides al sistema operativo que cree un hilo nativo, este tiene que reservarle obligatoriamente un bloque de memoria fijo llamado Stack (Pila). En la mayoría de sistemas operativos (como Linux o Windows), este tamaño por defecto suele ser de 2 Megabytes. Da igual si tu hilo solo va a sumar $2 + 2$ (que ocupa unos pocos bytes); el sistema operativo ya le ha amputado 2 MB a la memoria RAM de tu ordenador solo para gestionar ese hilo. Si intentas crear 10.000 hilos a la vez, tu servidor colapsará porque necesitará unos 20 GB de RAM solo en «peajes de existencia».
- **Asincronía (`async/await`):** Las «tareas» asíncronas no son hilos del sistema operativo; son gestionadas por Rust como pequeñas estructuras de datos (máquinas de estados, como vimos antes). El peaje de entrada aquí es de unos pocos Bytes o Kilobytes (el espacio justo para guardar las variables locales de esa función asíncrona). Si la tarea necesita calcular algo muy grande en el Heap, gastará más, pero si la tarea es pequeña, ocupa casi nada. Por eso un solo hilo de ejecución asíncrono puede gestionar 100.000 tareas web simultáneas en una Raspberry Pi sin despeinarse.

2. ¿Qué significa "en espacio de usuario"?

Un sistema operativo moderno divide la memoria del ordenador en dos zonas de seguridad totalmente aisladas:

- **Espacio de Núcleo (Kernel Space):** Es la zona VIP y ultra-protegida donde vive el corazón del Sistema Operativo. Solo el Kernel puede hablar directamente con la tarjeta de red, el disco duro o el procesador. Crear o destruir un hilo (`std::thread`) requiere que tu programa llame al Kernel, lo cual es lento porque exige un cambio de contexto de seguridad en la CPU.
- **Espacio de Usuario (User Space):** Es la zona donde se ejecutan tus programas normales (como tu navegador, tu juego o tu servidor de Rust).
 - Decir que el Runtime (como Tokio) gestiona la asincronía en «espacio de usuario» significa que es tu propio programa en Rust el que decide qué tarea va primero y cuál va después, mediante código normal y corriente de Rust. No tiene que pedirle permiso al Kernel del sistema operativo cada vez que cambia de una tarea a otra, lo que hace que cambiar entre miles de tareas asíncronas sea ridículamente rápido y eficiente.

3.2. ¿Cómo funciona `async` / `await` en Rust?

A diferencia de JavaScript o C#, donde la asincronía viene integrada con un motor interno y recolección de basura, en Rust la asincronía es de coste cero (Zero-Cost Abstraction) y no incluye un planificador por defecto.

El funcionamiento se basa en tres pilares: El Trait Future, Las Máquinas de Estados (`async`) y El Ejecutor (Runtime).

3.2.1. ¿Qué es un Trait?

Un Trait (que significa **rasgo o característica** en inglés) es simplemente una lista de tareas o habilidades que un tipo de datos promete saber hacer. En otros lenguajes de programación se le conoce como una Interfaz.

Decimos que un tipo de datos implementa un trait cuando posee todas las habilidades que componen el trait. El trait se implementa mediante código.

Veamos un ejemplo muy sencillo. Definimos el trait **Hablador** con un solo método, **hacer_sonido(&self)**. Los traits pueden contener cualquier número de métodos. Cuando se definen los métodos en el trait solo se pone su cabecera. Los tipos que implementen el trait tienen que dar código a esos metodos cuya cabecera heredan del trait.

En el ejemplo, los struct **Perro** y **Pato** implementan (con la sentencia **impl**) el trait **Hablador** y dan código al método **hacer_sonido**. Podemos ver en el main como se instancian los objetos perro y pato y llaman al método `hacer_sonido` de cada objeto.

```
/*
   --- Este es el contrato ---
   Cualquier tipo (de dato) que sea un "Hablador"
   debe saber hacer_sonido
*/

// Definimos el trait Hablador
trait Hablador {
    fn hacer_sonido(&self);
}

struct Perro {
    nombre: String,
}
```

```

struct Pato {
    nombre: String,
}

// El Perro firma el contrato "Hablador"
impl Hablador for Perro {
    fn hacer_sonido(&self) {
        println!("¡Guau! Soy el perro {}", self.nombre);
    }
}

// El Pato firma el contrato "Hablador"
impl Hablador for Pato {
    fn hacer_sonido(&self) {
        println!("¡Cuaic! Soy el pato {}", self.nombre);
    }
}

fn main() {
    let perro = Perro {
        nombre: String::from("Jerry"),
    };

    let pato = Pato {
        nombre: String::from("Lucas"),
    };

    perro.hacer_sonido();
    pato.hacer_sonido();
}

```

Gracias a este mecanismo, podemos crear funciones que no necesitan saber si tratan con un perro o con un pato; solo necesitan saber que el objeto que reciben cumple con el contrato de **saber hablar**.

De la misma forma, el **Trait Future** del que hablaremos después es simplemente un contrato que dice: «No me importa qué tipo de tarea asíncrona seas (red, disco, base de datos), si firmas este contrato, te obligas a ti mismo a tener una función llamada **poll** para que pueda preguntarte si ya has terminado».

3.2.2. A. El Trait Future (El modelo de «Tirar» o Poll)

En la mayoría de lenguajes, cuando lanzas una función asíncrona, esta empieza a ejecutarse inmediatamente en segundo plano (modelo Push). En Rust no.

Una función `async` devuelve un objeto que implementa el `trait Future`. Un `Future` es una estructura perezosa (lazy): no hace absolutamente nada hasta que alguien se lo pide. Su diseño simplificado es el siguiente:

```

pub trait Future {
    type Output;
    // El runtime llama a 'poll' para ver si el valor ya está listo
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}

```

El Runtime llama al método `poll()`. Este responde `Poll::Pending` (si sigue esperando el archivo o la red) o `Poll::Ready(valor)` (si ya terminó).

3.2.3. B. Qué hace `async` por dentro: La Máquina de Estados

Cuando marcas una función o bloque con la palabra clave `async`, el compilador de Rust transforma mágicamente tu código secuencial en una estructura de datos que actúa como una máquina de estados.

Cada vez que pones un `.await`, el compilador genera un punto de interrupción en esa máquina de estados.

```
// 1. Código escrito por el programador:
async fn procesar_pedido() -> String {
    let id = descargar_id().await; // Punto de pausa 1
    let resultado = verificar_pago(id).await; // Punto de pausa 2
    resultado
}
```

Por detrás, el compilador traduce ese código en un enum similar a esto:

```
// 2. Lo que genera el compilador (simplificado):
enum ProcesarPedidoFuture {
    Inicio,
    EsperandoDescargarId(DescargarIdFuture),
    EsperandoVerificarPago(VerificarPagoFuture),
    Completado,
}
```

Cuando la ejecución topa con un `.await`, el estado de la función se guarda en memoria (en el stack de la tarea, no del hilo) y la función retorna el control inmediatamente.

3.2.4. C. El Rol del Runtime (Ej. Tokio o el de Macroquad)

Dado que la biblioteca estándar de Rust no incluye un planificador asíncrono para mantener los binarios lo más pequeños posibles, necesitas un Runtime externo (como Tokio para servidores o el interno de Macroquad para juegos).

- El Runtime mantiene una cola de Futures.
- Llama a `poll()` en el Future.
- Si el Future devuelve `Poll::Pending`, el Runtime lo registra en el sistema operativo (usando herramientas eficientes como `epoll` en Linux o `IOCP` en Windows) para que el hardware le avise cuando haya actividad en la tarjeta de red o el disco.
- Mientras tanto, ese hilo del Runtime pasa a procesar otro Future de la cola.
- Cuando el S.O. avisa que los datos llegaron, el Runtime despierta al Future correspondiente mediante un componente llamado `Waker` y vuelve a llamar a `poll()`, avanzando al siguiente estado.

[ESTO LO PONGO AQUÍ PARA NO PERDERLO PERO ES UNA CONSULTA A LA IA]

Yo: ¿Por qué llamas a Macroquad Runtime interno?

Llamamos a Macroquad «runtime interno» porque el propio motor del juego incluye e implementa su propio planificador asíncrono dentro de su código base, sin obligarte a depender de herramientas externas como Tokio o `async-std`.

En Rust, la biblioteca estándar provee la sintaxis (`async`, `await`) y la interfaz base (`Future`), pero no incluye el motor que hace rodar esos futuros. Alguien tiene que encargarse de llamar a `poll()` y gestionar la cola de tareas.

Macroquad necesita controlar de forma milimétrica el tiempo y los fotogramas del juego. Por ello, en lugar de usar un runtime de propósito general (como Tokio, que está diseñado para servidores web y bases de datos), implementa su propio bucle de control asíncrono optimizado para gráficos:

1. Gestión del bucle gráfico: El macro-bucle de Macroquad toma tu función `async fn main()` y ejecuta su código frame a frame.
2. Control del `.await`: Cuando tu código llega a `next_frame().await`, el planificador interno de Macroquad sabe exactamente que debe pausar tu función `main`, cederle el control a la tarjeta gráfica para que dibuje la pantalla y, justo en el siguiente fotograma, volver a despertar tu función para continuar con la lógica.

En resumen, se le llama «interno» porque viene dentro de la propia caja de Macroquad, permitiendo que los juegos en Rust funcionen de forma asíncrona con una configuración cero para el programador.

AQUI TERMINA LA CONSULTA

3.3. Ejemplo práctico: Integración en nuestro videojuego

— [No hay un apartado anterior con Macroquad] —

En el apartado anterior de Macroquad usamos `next_frame().await`. Ahora cobra sentido:

```
#[macroquad::main("Mi Juego Async")]
async fn main() {
    loop {
        // ... Lógica del juego ...

        // Detiene la máquina de estados de 'main' y le cede el control
        // al motor gráfico para que dibuje. Cuando la tarjeta gráfica
        // termina el frame, el runtime vuelve a despertar este bucle.
        next_frame().await;
    }
}
```

3.4. Concurrencia Segura - El Superpoder de Rust

3.5. ¿Qué es la concurrencia y por qué es difícil? El problema de los hilos de ejecución.

(De antes)

Normalmente, tus programas ejecutan una línea de código detrás de otra (secuencialmente). Sin embargo, los procesadores modernos tienen varios núcleos y pueden hacer varias tareas a la vez. Esto es la concurrencia.

Hacer esto en otros lenguajes es peligroso: si dos hilos intentan modificar la misma variable al mismo tiempo, el programa puede corromperse (lo que se conoce como condición de carrera). El compilador de Rust, gracias a sus reglas de propiedad (`ownership`), impide estos

errores antes de que el programa se ejecute. A esto le llamamos Concurrency sin miedo (Fearless Concurrency).

(Lo nuevo)

La concurrencia es la capacidad de un programa de ejecutar diferentes partes de su código de forma independiente y potencialmente simultánea. Tradicionalmente, esto se logra mediante hilos de ejecución (threads) gestionados por el sistema operativo. El gran problema de la concurrencia en lenguajes como C++ o Java es que los hilos comparten memoria por defecto.

Cuando varios hilos intentan acceder y modificar el mismo fragmento de datos al mismo tiempo sin la sincronización adecuada, ocurren condiciones de carrera (race conditions), corrupción de memoria y bloqueos mutuos (deadlocks). Estos errores son extremadamente difíciles de reproducir y depurar porque dependen del orden exacto en el que el sistema operativo planifica los hilos.

A continuación, se muestra una simulación conceptual del problema común de «lectura/escritura sucia» si no se tuviera el control estricto de Rust:

```
// Nota: Este patrón simula el peligro lógico de la concurrencia clásica.
// En otros lenguajes, si dos hilos modifican `cuenta_bancaria` a la vez, el resultado
// final es impredecible.

struct CuentaBancaria {
    balance: i32,
}

impl CuentaBancaria {
    fn retirar(&mut self, cantidad: i32) {
        let saldo_actual = self.balance;
        // Si otro hilo interrumpe aquí, ambos leerán el mismo saldo_actual
        self.balance = saldo_actual - cantidad;
    }
}
```

3.6. 2.3. Creando hilos (std::thread)

Podemos lanzar un «hilo» secundario para que haga un trabajo pesado mientras el programa principal sigue su curso.

3.7. El enfoque de Rust: Fearless Concurrency (Concurrency sin miedo).

El superpoder de Rust radica en su concepto de Fearless Concurrency (Concurrency sin miedo). En lugar de delegar la prevención de errores al programador mediante documentación y disciplina, el propio compilador de Rust valida la seguridad concurrente en tiempo de compilación.

- Rust logra esto extendiendo sus reglas de propiedad (ownership) y préstamo (borrowing) mediante dos traits marcadores del sistema de tipos:
- Send: Indica que la propiedad de un tipo puede transferirse entre hilos.
- Sync: Indica que es seguro que múltiples hilos accedan al mismo tipo a través de referencias compartidas (&T).

Si intentas escribir código que cause una condición de carrera, el código simplemente no compilará.

El siguiente ejemplo demuestra cómo el compilador bloquea un intento de compartir una referencia insegura entre hilos:

```
use std::thread;

fn main() {
    let mut datos = vec![1, 2, 3];

    // Intentamos crear un hilo que acceda a 'datos' de forma insegura
    // El compilador arrojará un error porque no puede garantizar la validez de la
    referencia
    /*
    thread::spawn(|| {
        datos.push(4);
    });

    datos.push(5); // Error: Uso de un valor mutabilidad compartida entre hilos
    */
    println!("El compilador de Rust nos protege antes de ejecutar el programa.");
}
```

3.8. Creando y gestionando hilos (std::thread)

Para crear un hilo nativo en Rust se utiliza la función `thread::spawn` de la biblioteca estándar, pasándole una clausura (closure) que contiene el código que debe ejecutar el hilo.

Debido a que los hilos pueden sobrevivir a la función que los creó, Rust nos obliga a transferir la propiedad de las variables del entorno al hilo mediante la palabra clave `move`. Para asegurarnos de que un hilo termine su ejecución antes de que el hilo principal finalice, guardamos el manejador devuelto (`JoinHandle`) y llamamos al método `.join()`.

Ejemplo práctico de creación, paso de datos y sincronización de hilos:

```
use std::thread;
use std::time::Duration;

fn main() {
    let mensaje = String::from("Hola desde el hilo secundario");

    // Creamos un hilo. Usamos 'move' para transferir la propiedad de 'mensaje' al hilo
    let manejador = thread::spawn(move || {
        for i in 1..4 {
            println!("{}", (Paso {})", mensaje, i);
            thread::sleep(Duration::from_millis(200));
        }
    });

    // El hilo principal hace su propio trabajo en paralelo
    for i in 1..3 {
        println!("Trabajando desde el hilo principal... (Paso {})", i);
        thread::sleep(Duration::from_millis(150));
    }

    // Esperamos obligatoriamente a que el hilo secundario termine
    manejador.join().unwrap();
}
```

```
println!("{}", Todos los hilos han terminado!);
}
```

3.9. Compartiendo datos de forma segura: Canales de comunicación (mpsc)

Una de las filosofías de concurrencia más populares (adoptada también por lenguajes como Go) es: «No te comuniques compartiendo memoria; en su lugar, comparte memoria comunicándote». Rust implementa esto mediante canales MPSC (Multi-Producer, Single-Consumer o Múltiples Productores, Un Solo Consumidor).

El módulo `std::sync::mpsc::channel` devuelve una tupla con dos elementos: un transmisor (Sender) y un receptor (Receiver). Podemos clonar el transmisor para enviar mensajes desde múltiples hilos concurrentes hacia un único hilo central que procesa los resultados de manera segura y ordenada.

Ejemplo práctico utilizando un canal MPSC con múltiples productores:

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // Creamos el canal de comunicación
    let (tx, rx) = mpsc::channel();

    // Clonamos el transmisor para tener un segundo productor
    let tx1 = tx.clone();

    // Hilo Productor 1
    thread::spawn(move || {
        let mensajes = vec![String::from("uno"), String::from("dos")];
        for msg in mensajes {
            tx1.send(msg).unwrap(); // Envía el mensaje por el canal
            thread::sleep(Duration::from_millis(200));
        }
    });

    // Hilo Productor 2
    thread::spawn(move || {
        let mensajes = vec![String::from("tres"), String::from("cuatro")];
        for msg in mensajes {
            tx.send(msg).unwrap(); // Envía el mensaje por el canal
            thread::sleep(Duration::from_millis(200));
        }
    });

    // Hilo Consumidor (Hilo principal)
    // El iterador del 'rx' se bloquea esperando mensajes hasta que todos los 'tx' se destruyen
    for recibido in rx {
        println!("Mensaje recibido de forma segura: {}", recibido);
    }

    println!("Canal cerrado. Todos los datos fueron procesados.");
}
```

3.10. 2.4. Canales de comunicación (mpsc)

¿Cómo se pasan información los hilos? Rust utiliza canales. mpsc significa Multiple Producer, Single Consumer (Múltiples productores, un solo consumidor). Imagina que es un tubo donde varios hilos pueden meter mensajes, pero solo uno los recibe al final.

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // Creamos un canal. 'tx' es el transmisor (envía), 'rx' es el receptor (recibe)
    let (tx, rx) = mpsc::channel();

    // Lanzamos un hilo que calculará algo y enviará el resultado
    thread::spawn(move || {
        let resultado_calculo = "Resultado ultra secreto del sensor";
        thread::sleep(Duration::from_secs(2)); // Simulamos un trabajo largo
        tx.send(resultado_calculo).unwrap(); // Enviamos el dato
    });

    println!("Esperando los datos del hilo secundario...");

    // El hilo principal se bloquea de forma segura hasta que llegue el mensaje
    let mensaje_recibido = rx.recv().unwrap();
    println!("Recibido con éxito: {}", mensaje_recibido);
}
```

4. Proyecto Final de Ciclo - Tu Primer Videojuego Gráfico

Para este proyecto utilizaremos Macroquad, una librería de Rust increíblemente ligera que no requiere configuraciones complejas de sistemas operativos ni herramientas pesadas, ideal para los ordenadores del instituto.

Nota de configuración para el Cargo.toml: ?????

4.1. Introducción al desarrollo de videojuegos con Macroquad.

Macroquad es una biblioteca minimalista y extremadamente ligera para el desarrollo de videojuegos en Rust. Su principal ventaja es que elimina la configuración compleja (boilerplate) y no requiere pesadas dependencias del sistema operativo, permitiendo compilar fácilmente para escritorio (Windows, Mac, Linux), móviles y entornos web (WebAssembly).

A diferencia de otros motores que imponen patrones arquitectónicos rígidos, Macroquad utiliza funciones directas y globales para renderizar figuras, procesar la entrada del teclado o ratón y gestionar el tiempo, lo que la convierte en la herramienta ideal para estudiantes que dan sus primeros pasos en el desarrollo gráfico con Rust.

Para empezar, solo necesitas añadir macroquad = «0.4» (o la versión actual) a tu archivo Cargo.toml. A continuación se muestra la estructura mínima para abrir una ventana:

cargo add macroquad

```
use macroquad::prelude::*;

#[macroquad::main("Mi Primer Juego")]
async fn main() {
    // Aquí inicializaremos las variables de nuestro juego antes de arrancar
    let color_fondo = LIGHTGRAY;

    loop {
        // Limpiamos la pantalla en cada fotograma
        clear_background(color_fondo);

        // Dibujamos un círculo azul en el centro de la pantalla
        draw_circle(screen_width() / 2.0, screen_height() / 2.0, 50.0, BLUE);

        // Sincroniza con el refresco de la pantalla (espera al siguiente frame)
        next_frame().await
    }
}
```

4.2. El bucle principal de un juego (Game Loop)

El Game Loop (Bucle de Juego) es el corazón de cualquier videojuego. Se trata de un ciclo infinito que se ejecuta continuamente (idealmente 60 veces por segundo o más) y realiza tres tareas fundamentales en un orden estricto: Leer entradas (mandos, teclado), Actualizar lógica (mover personajes, calcular colisiones) y Dibujar en pantalla.

Macroquad gestiona este bucle mediante una función asíncrona (async/await) combinada con un ciclo loop. La llamada a `next_frame().await` al final del bucle es crucial: detiene la ejecución temporalmente para ceder el control al hardware gráfica, asegurando que el juego no consuma el 100% de la CPU de forma descontrolada y manteniendo una tasa de fotogramas suave y constante.

El siguiente ejemplo muestra cómo estructurar estas tres fases integrando el movimiento por teclado:

```
use macroquad::prelude::*;

#[macroquad::main("El Bucle de Juego")]
async fn main() {
    let mut x = screen_width() / 2.0;
    let y = screen_height() / 2.0;
    let velocidad = 5.0;

    loop {
        // 1. LEER ENTRADAS & 2. ACTUALIZAR LÓGICA
        if is_key_down(KeyCode::Left) {
            x -= velocidad;
        }
        if is_key_down(KeyCode::Right) {
            x += velocidad;
        }

        // 3. DIBUJAR
        clear_background(BLACK);

        draw_text("Mueve el cuadrado con izquierda/derecha", 20.0, 30.0, 20.0, WHITE);
        draw_rectangle(x - 20.0, y - 20.0, 40.0, 40.0, RED);

        next_frame().await
    }
}
```

4.3. Desarrollo Guiado: Clon de Pong / Rust-Bird

Para consolidar lo aprendido, desarrollaremos una versión simplificada del clásico Pong. El juego requiere estructurar la lógica mediante el control de una pala (jugador), el movimiento automático de la pelota y la detección matemática de colisiones básicas (cajas delimitadoras de la pala y límites de la pantalla).

Este proyecto práctico demuestra cómo la seguridad y velocidad de Rust operan en un entorno interactivo en tiempo real:

```
use macroquad::prelude::*;

struct Pala {
    x: f32,
    y: f32,
    ancho: f32,
    alto: f32,
    velocidad: f32,
}

struct Pelota {
    x: f32,
```

```

    y: f32,
    radio: f32,
    vel_x: f32,
    vel_y: f32,
}

#[macroquad::main("Rust Pong Simplificado")]
async fn main() {
    let mut jugador = Pala {
        x: 50.0,
        y: screen_height() / 2.0 - 40.0,
        ancho: 15.0,
        alto: 80.0,
        velocidad: 7.0,
    };

    let mut pelota = Pelota {
        x: screen_width() / 2.0,
        y: screen_height() / 2.0,
        radio: 8.0,
        vel_x: 5.0,
        vel_y: 5.0,
    };

    loop {
        // Entrada del Jugador
        if is_key_down(KeyCode::Up) && jugador.y > 0.0 {
            jugador.y -= jugador.velocidad;
        }
        if is_key_down(KeyCode::Down) && jugador.y < screen_height() - jugador.alto {
            jugador.y += jugador.velocidad;
        }

        // Movimiento de la pelota
        pelota.x += pelota.vel_x;
        pelota.y += pelota.vel_y;

        // Rebote en techo y suelo
        if pelota.y - pelota.radio < 0.0 || pelota.y + pelota.radio > screen_height() {
            pelota.vel_y = -pelota.vel_y;
        }

        // Rebote en pared derecha (Simulación de rival)
        if pelota.x + pelota.radio > screen_width() {
            pelota.vel_x = -pelota.vel_x;
        }

        // Colisión simple con la pala del Jugador
        if pelota.x - pelota.radio <= jugador.x + jugador.ancho
            && pelota.y >= jugador.y
            && pelota.y <= jugador.y + jugador.alto
        {
            pelota.vel_x = -pelota.vel_x;
            pelota.x = jugador.x + jugador.ancho + pelota.radio; // Evita atascarse
        }

        // Reinicio si sale por la izquierda (Derrota)
        if pelota.x < 0.0 {
            pelota.x = screen_width() / 2.0;
            pelota.y = screen_height() / 2.0;
            pelota.vel_x = 5.0;
        }

        // Renderizado
    }
}

```

```

        clear_background(DARKGRAY);

        // Dibujar pala y pelota
        draw_rectangle(jugador.x, jugador.y, jugador.anch, jugador.alto, WHITE);
        draw_circle(pelota.x, pelota.y, pelota.radio, YELLOW);

        next_frame().await
    }
}

```

4.4. 3.3. Desarrollo Guiado: Un clon de Pong simplificado

Vamos a programar una versión básica de Pong donde controlamos una barra que debe rebotar una pelota para evitar que caiga. Código del juego (src/main.rs):

```

use macroquad::prelude::*;

#[macroquad::main("Mi primer juego en Rust")]
async fn main() {
    // Variables de la pelota
    let mut bola_x = screen_width() / 2.0;
    let mut bola_y = screen_height() / 2.0;
    let mut velocidad_x = 4.0;
    let mut velocidad_y = 4.0;
    let radio_bola = 15.0;

    // Variables de la barra del jugador
    let ancho_barra = 120.0;
    let alto_barra = 20.0;
    let mut barra_x = (screen_width() - ancho_barra) / 2.0;
    let barra_y = screen_height() - 40.0;
    let velocidad_barra = 7.0;

    let mut puntuacion = 0;

    // --- EL BUCLE PRINCIPAL del juego ---
    loop {
        // 1. Entrada de usuario
        if is_key_down(KeyCode::Left) && barra_x > 0.0 {
            barra_x -= velocidad_barra;
        }
        if is_key_down(KeyCode::Right) && barra_x < screen_width() - ancho_barra {
            barra_x += velocidad_barra;
        }

        // 2. Actualizar lógica del juego
        bola_x += velocidad_x;
        bola_y += velocidad_y;

        // Rebote en paredes laterales
        if bola_x - radio_bola < 0.0 || bola_x + radio_bola > screen_width() {
            velocidad_x = -velocidad_x;
        }
        // Rebote en el techo
        if bola_y - radio_bola < 0.0 {
            velocidad_y = -velocidad_y;
        }

        // Colisión con la barra del jugador
        if bola_y + radio_bola >= barra_y
            && bola_x >= barra_x
            && bola_x <= barra_x + ancho_barra
    }
}

```

```

    {
        velocidad_y = -velocidad_y;
        puntuacion += 1;
    }

    // Condición de derrota (La pelota cae al fondo)
    if bola_y > screen_height() {
        // Reiniciar juego
        bola_x = screen_width() / 2.0;
        bola_y = screen_height() / 2.0;
        puntuacion = 0;
    }

    // 3. Dibujar en la pantalla
    clear_background(DARKGRAY);

    // Dibujar Pelota
    draw_circle(bola_x, bola_y, radio_bola, WHITE);

    // Dibujar Barra
    draw_rectangle(barra_x, barra_y, ancho_barra, alto_barra, RED);

    // Dibujar Puntuación
    draw_text(&format!("Puntos: {} ", puntuacion), 20.0, 40.0, 30.0, GREEN);

    // Espera al siguiente fotograma (mantiene los 60 FPS)
    next_frame().await
}
}

```

5. Ejercicios propuestos

Para evaluar la asimilación del contenido, se le propone al estudiante modificar el código base añadiendo alguna de estas características:

- Incremento de dificultad: Hacer que la velocidad de la pelota aumente gradualmente cada vez que golpee la barra.
- Integración Web/Concurrente (Opcional): Usar hilos para guardar la puntuación más alta en un archivo en segundo plano sin congelar los gráficos del juego.

6. Desafíos propuestos para el alumno (Personalización e integración)

Una vez que el motor base funciona de forma estable, el alumno debe aplicar los conceptos avanzados estudiados a lo largo de los cuadernos de Rust para expandir las capacidades del videojuego.

Se proponen los siguientes tres desafíos individuales:

- Sistema de Marcador Estático: Crea una variable mutable o estructura para llevar el conteo de los puntos. Utiliza `draw_text` para mostrar la puntuación en la parte superior de la pantalla y aumenta el contador cada vez que la pelota rebote con éxito en la pala del jugador.
- Enemigo Automatizado (IA Simple): Añade una segunda pala en el extremo derecho de la pantalla controlado por el juego. Puedes programar una lógica básica que compare la coordenada y de la pelota con la coordenada y de la pala enemiga para que esta se desplace hacia arriba o hacia abajo intentando interceptarla.
- Concurrencia para Efectos de Sonido o Logs (Opcional Avanzado): Utiliza los conceptos de canales (mpsc) aprendidos en los módulos previos para enviar mensajes desde el bucle principal hacia un hilo secundario de logs cada vez que ocurra una colisión o se marque un punto, evitando que las operaciones de entrada/salida bloqueen el flujo gráfico principal.

```
// Pista para el alumno (Estructura del marcador)
struct Marcador {
    puntos_jugador: u32,
    puntos_ia: u32,
}

// Ejemplo de cómo pintar el texto en el bucle principal:
// draw_text(&format!("{}", marcador.puntos_jugador), 40.0, 40.0, 30.0, WHITE);
```

¿Deseas que preparemos la plantilla de soluciones detalladas para alguno de los desafíos propuestos (como la IA del rival o el sistema de puntuación), o prefieres dar por cerrado el temario del Cuaderno 5? Para entender los programas de este cuaderno tenemos que añadir algunas aclaraciones teóricas:

- Closures, Lambdas, funciones anónimas, callbacks
- `async / await`

```
// Aclarar:

thread::spawn(|| {
    datos.push(4);
});
```